

AI Engineering

Handwritten Revision Notes

10 Topics · Last-minute Revision · Exam Ready

Inside

1. Introduction to AI Engineering
2. Modern Text & Image Architectures
3. LLM APIs & Decoding Strategies
4. Evaluation Pipelines & Observability
5. Prompt Engineering
6. Retrieval-Augmented Generation
7. Embeddings Deep Dive
8. Multimodal RAG
9. Prompt Engineering Security
10. AI Agents — Foundations & Planning

★ **2-Page Ultra-Condensed Summary at the end**

Topic 1 Introduction to AI Engineering

1 Core Idea — "What is it?"

AI Engineering = building **production applications** powered by **pretrained foundation models** (LLMs, diffusion, multimodal). You don't train — you orchestrate.

2 Why it matters

- Foundation models shifted the bottleneck → from **training** to **integration**
- Devs now compose, prompt, evaluate & deploy intelligence
- Fast iteration → product-led, not research-led

3 Types of Foundation Models

- **LLMs** → text (GPT, Claude, Llama, Mistral)
- **Diffusion** → images (Stable Diffusion, DALL-E, Midjourney)
- **Multimodal** → text + image + audio (GPT-4o, Gemini, Claude)

4 Core Responsibilities of an AI Engineer

- Integrate pretrained foundation models into apps
- **Prompt** design & optimization
- **Orchestration** of AI components (chains, tools, routers)
- Build **RAG, Agents, Tool-use** systems
- **Evaluation** & observability pipelines
- **Fine-tuning** when prompting isn't enough
- Production: latency · cost · reliability · safety

5 The 5-Layer Application Stack

1. APPLICATION LAYER	← UI, chatbot, API endpoint
2. ORCHESTRATION LAYER	← LangChain, prompts, tools
3. OBSERVABILITY LAYER	← logs, traces, evals
4. MODEL LAYER	← LLM / embedding / vision
5. INFRASTRUCTURE LAYER	← GPUs, vector DB, cloud

6 ⚙️ ML Engineering vs AI Engineering

<i>ML Engineering</i>	<i>AI Engineering</i>
Trains models from raw data	Uses pretrained foundation models
Data-heavy, label-driven	Prompt & context-driven
Slow iteration (days/weeks)	Fast iteration (minutes/hours)
Stats, ML, MLOps skills	Prompting, APIs, RAG, system design
Huge compute (training GPUs)	Mostly inference cost
Accuracy, F1, AUC metrics	LLM-as-judge, BLEU, human eval
Concerns: drift, retraining	Concerns: hallucination, cost, safety

7 🌐 Ecosystem Snapshot

- **Model providers:** OpenAI, Anthropic, Google, Meta, Mistral
- **Vector DBs:** Pinecone, Weaviate, Qdrant, Chroma, Milvus
- **Eval tools:** Ragas, DeepEval, TruLens, Promptfoo
- **Observability:** LangSmith, Langfuse, Arize, Helicone
- **Frameworks:** LangChain, LlamaIndex, DSPy, Haystack
- **Infra:** AWS Bedrock, Azure OpenAI, Modal, Replicate

 Quick Revision Takeaway

AI Engineer = bridges pretrained models + real-world products. **You don't train**
— **you orchestrate**. Integration, prompts & evals are the core skills.

Topic 2 Modern Text & Image Architectures

1 Core Idea

Modern LLMs = **decoder-only Transformers** trained to predict the **next token**.
Attention is what makes it work.

2 The Journey

RNN/LSTM → Encoder-Decoder → Decoder-Only 🚀

► Encoder-Decoder (original 2017)

- **Encoder** — reads input fully (bidirectional)
- **Decoder** — generates output one token at a time
- Great for seq2seq → translation, summarization

► Decoder-Only (modern LLMs)

- Single stack predicts the **next token**
- **Why it won:**
 - Training simplicity — one objective
 - Scales beautifully
 - Emergent abilities at scale
 - Unified interface — every task = text-in / text-out

3 Decoder-Only Data Flow

```
Input text
↓ 1. Tokenization (BPE / SentencePiece)
↓ 2. Embedding lookup
↓ 3. + Positional information
↓ 4. N × Transformer blocks
↓ 5. Output projection (linear)
↓ 6. Softmax → probability over vocab
↓ 7. Pick next token → repeat
```

4 Positional Encoding

- **Why?** Self-attention is order-blind by default

- **Learned absolute** — each position = trainable vector
- **RoPE** (Rotary Position Embedding) — rotates Q,K vectors → relative position info, used in Llama/Mistral

5 Self-Attention

Intuition

Every token asks: "who should I pay attention to?"

Q = Query (what I'm looking for)

K = Key (what I contain)

V = Value (what I'll share)

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q} \cdot \mathbf{K}^T / \sqrt{d_k}) \cdot \mathbf{V}$$

Scale by $\sqrt{d_k}$ → keeps softmax stable

6 Causal Masking

- Decoder-only models must **NOT see future tokens**
- Add a mask ($-\infty$) to upper-triangle of $\mathbf{Q} \cdot \mathbf{K}^T$ **before softmax**
- Result: token i only attends to tokens $\leq i$
- **Contrast:** BERT = bidirectional → encoder model

7 Multi-Head Attention

- One head learns **one type of relation** (syntax, coref, ...)
- Multiple heads → capture multiple relations in parallel

Split → attend per head → concat → linear projection

Modern upgrade — GQA

Grouped-Query Attention: heads share K,V → smaller **KV cache**, faster inference. Used in Llama 2/3, Mistral.

8 Transformer Block (Pre-Norm)

$x \rightarrow \text{LayerNorm} \rightarrow \text{Masked Self-Attn} \rightarrow + \text{residual}$
 $\downarrow$
 $\rightarrow \text{LayerNorm} \rightarrow \text{FFN} \rightarrow + \text{residual} \rightarrow \text{out}$

- **Pre-norm** (LN before sub-layer) = standard $\rightarrow$ stabler training at scale

9 *Tiny Attention Example*

Tokens: [The, cat, sat]

Step 1 $\rightarrow$ raw scores $Q \cdot K^T$: [3.0, 1.5, 0.5]

Step 2 $\rightarrow$ scale $\div \sqrt{d_k}$ ($\sqrt{2} \approx 1.41$): [2.13, 1.06, 0.35]

Step 3 $\rightarrow$ softmax: [0.65, 0.23, 0.12]

Step 4 $\rightarrow$ weighted sum of V vectors $\rightarrow$ context vector
 $\Rightarrow$ "sat" attends most to "The"

Quick Revision Takeaway

Decoder-only Transformer = embeddings + position + (masked self-attn + FFN) $\times$ N + softmax. **Attention is the core. Causal mask makes it generative.**

Topic 3 *LLM APIs & Decoding Strategies*

1 *Core Idea*

LLM API = HTTP endpoint that returns next-token predictions. **Decoding strategy** decides HOW to pick the next token from the probability distribution.

2 *Open-source vs Third-party APIs*

<i>Open-source (self-host)</i>	<i>Third-party API</i>
Llama, Mistral, Qwen	OpenAI, Anthropic, Gemini
Full control, privacy	Easy, no infra
You manage infra / cost	Pay per token
Custom fine-tunes	Latest frontier models

3 *How generation works*

prompt → logits → softmax → probability → **DECODE** → token → append → repeat

4 *Decoding Strategies*

1. Greedy Search

Always pick the highest-probability token.

Example: "She loves to ___" → "**read**" (top-1)

✓ Fast, deterministic ✗ Repetitive, dull, can loop

2. Beam Search

Keeps top-k **whole sequences** alive at each step.

Picks the sequence with best total probability.

✓ Better for translation, structured output ✗ Slow, bland for open generation

3. Top-k Sampling

Restrict to top **k** tokens, sample from them.

Example: $k=5 \rightarrow$ sample from 5 most likely words

✓ Balance of diversity & quality ✗ Fixed $k \rightarrow$ too narrow or too wide

4. Top-p (Nucleus) Sampling

Keep smallest set of tokens whose total prob $\geq p$.

Example: $p=0.9 \rightarrow$ adapts vocab dynamically

✓ Adaptive, very popular ✗ Tricky to tune

5. Temperature 📉

Applied to logits **BEFORE** softmax: $\text{logits} \div T$

Low T ($\rightarrow 0$) $\rightarrow$ sharper distribution $\rightarrow$ more deterministic

High T ($\rightarrow 1.5+$) $\rightarrow$ flatter distribution $\rightarrow$ more creative

softmax(logits / T)

$T \downarrow$ confident | $T \uparrow$ creative

Mini calculation — logits = [2.0, 1.0, 0.5]

T=1.0 $\rightarrow$ softmax $\approx$ [0.59, 0.22, 0.13] (normal)

T=0.5 $\rightarrow$ softmax $\approx$ [0.79, 0.15, 0.06] (sharper)

T=2.0 $\rightarrow$ softmax $\approx$ [0.46, 0.29, 0.22] (flatter)

5



Comparison Table

<i>Method</i>	<i>Type</i>	<i>Diversity</i>	<i>Quality</i>	<i>Typical use</i>
Greedy	Determ.	Low	Medium	Code, math
Beam	Determ.	Low	High	Translation
Top-k	Stoch.	Medium	Good	Chat
Top-p	Stoch.	High	Good	Creative writing
Temp	Knob	Tunable	Tunable	Used WITH top-k/p



Practical Note

Real systems **combine**: temperature + top-p (most common) OR temperature + top-k . Pure greedy is rare except for structured output.



Quick Revision Takeaway

Greedy = boring. Beam = structured. Top-k/Top-p = diverse. **Temperature = creativity dial.** Combine them!

Topic 4 *Evaluation Pipelines & Observability*

1 *Core Idea*

Evaluation = how you know your LLM app actually works. **Without it, you're flying blind in production.**

2 *Types of LLM Systems*

- **Single-prompt** — one input, one model call
- **RAG** — retrieve + generate
- **Agentic** — multi-step, tools, decisions

3 *Key Questions Eval Answers*

- How well is the app working?
- Does it handle edge scenarios?
- Can it be improved?
- Is it safe to deploy?

4 *⚠ Why LLM Evals are HARD*

- **Non-deterministic** — same input $\neq$ same output
- No single "correct" answer
- Wide input space $\rightarrow$ infinite test cases
- **Risks:** hallucination, prompt injection, data leaks, bias, toxicity

5 *⚖ Model Eval vs Product Eval*

<i>Model Eval</i>	<i>Product Eval</i>
Tests model capability	Tests full system performance
MMLU, HumanEval, HellaSwag	Task-specific user metrics
"Is the model smart?"	"Does the app work for users?"
Done by model providers	Done by AI engineers

6 *Evaluation Methods*

► *Reference-based*

- **BLEU** → precision-focused n-gram overlap (translation)
- ex: pred='The cat sat' vs ref='The cat sat down' → high BLEU
- **ROUGE** → recall-focused (summarization). ROUGE-1 = unigrams, ROUGE-2 = bigrams, ROUGE-L = longest common subseq
- **BERTScore** → semantic similarity via embeddings + cosine. Captures meaning, not just words

► *Reference-free / LLM-as-Judge*

- Use a **strong LLM** to grade outputs
- **Pairwise comparison** → A vs B, which is better?
- **Criteria-based (no ref)** → rate relevance, politeness 1–5
- **Criteria-based (with ref)** → compare to gold answer
- Use cases: relevance, hallucination, politeness, moderation

7 *Observability / Online Evals*

- 👍 User feedback (thumbs up/down)
- 📈 Engagement (session length, retention)
- ✅ Task effectiveness (did the user succeed?)
- ⌚ Latency (P50, P95, P99)
- ❌ Error rate, timeouts, refusals
- 💰 Business metrics (revenue, conversions, cost/query)

8 *Eval Flow*

Dataset → System → Outputs → Metrics → Analysis → Iterate

Experiment Tracking

Track every prompt version, model, params, eval result.

Tools: **LangSmith, Langfuse, Weights & Biases, MLflow**

Quick Revision Takeaway

Offline evals (BLEU/ROUGE/BERTScore/LLM-judge) before launch + **online evals** (feedback, latency, business) after launch. Both are needed.

Topic 5 Prompt Engineering

1 Core Idea

Prompt = **bridge between human intent and AI understanding**. A good prompt is a clear specification of the task.

2 *A prompt can be*

- A **question** → "What is the capital of France?"
- An **instruction** → "Translate this to Spanish"
- A **partial text** → "Once upon a time..."
- A **complex task** → "Analyze this contract and flag risks"

3 *Anatomy of a Prompt*

- 1. TASK** → what you want done
- 2. CONTEXT** → background, examples, persona
- 3. OUTPUT SPEC** → format, length, tone

4 *Simple / Direct Prompting*

- Ask directly, no reasoning steps
- Examples:
 - "Capital of Japan?" → "Tokyo"
 - "Translate hello to French" → "bonjour"
 - "Summarize this article in 3 lines"
- **Best for:** factual recall, simple tasks

5 *Chain of Thought (CoT)*

- Make the model **REASON step by step**
- Trigger phrase: "Let's think step by step."
- Human analogy: break problem → analyze → apply logic → conclude

Example

Q: Roger has 5 balls. He buys 2 cans of 3 balls each. How many?

CoT: Roger starts with 5. $2 \text{ cans} \times 3 = 6$. Total = $5 + 6 = 11$.

Answer: 11 ✓

6 **3** *ReAct = Reason + Act*

- Combines reasoning with **tool calling**
- Loop: **Think** → **Act (tool)** → **Observe** → **Think** → ...

<i>Chain of Thought</i>	<i>ReAct</i>
Reasoning only (in-head)	Reasoning + external tools
No real-world data	Can fetch live data
Math, logic puzzles	Search, APIs, agents

► *ReAct Flow — Weather Example*

```
User: "What's the weather in Paris?"
↓
Thought: need current weather → use weather API
↓
Action: weather_api(city="Paris")
↓
Observation: 18°C, cloudy
↓
Thought: I have the info
↓
Final Answer: "It's 18°C and cloudy in Paris."
```

7 **4** *Prompting for RAG*

- **Retrieve** relevant docs from knowledge base
- **Augment** the prompt with retrieved context
- **Generate** the answer grounded in that context

Open-book Exam Analogy

Instead of memorizing, the model "looks up" the answer from a textbook (your docs).

► *RAG Prompt Structure*

```
— INSTRUCTIONS —  
Answer using ONLY context below  
Cite sources. Say "idk" if N/A  
— CONTEXT (retrieved chunks) —  
[Chunk 1] ...  
[Chunk 2] ...  
— USER QUERY —  
"What is the refund policy?"
```

► *Best Practices*

- Use **only** the provided context
- Cite sources
- Handle insufficient context → say "I don't know"
- Respect token limits
- Structure context clearly (delimiters, headings)

Quick Revision Takeaway

Simple → CoT (think) → ReAct (think + act) → RAG (think + retrieve + answer).

Pick the smallest tool that works.

Topic 6 Retrieval-Augmented Generation (RAG)

1 Core Idea

RAG = **retrieve relevant documents** at query time + feed to LLM as context.
Solves knowledge gaps without retraining.

2 Why RAG exists

- 📅 **Knowledge cutoff** — model doesn't know recent info
- 👻 **Hallucination** — model invents facts
- 🔒 **Proprietary data** — model never saw your private docs
- 📦 **Context window limits** — can't fit entire knowledge base

3 RAG Flow

User Query → Retrieve docs → Augment prompt → Generate grounded answer

4 ⚖️ Approaches Compared

Approach	Cost	Updateable	Scalable
Prompt only	Low	Manual	Limited (token cap)
Fine-tuning	High	Hard	Static knowledge
RAG	Medium	Easy ✓	Highly ✓

5 ✂️ Chunking

- Split docs into **smaller pieces** for retrieval
- Why?** Embeddings work best on semantic units
- Too large** → noisy, expensive, low precision
- Too small** → no context, broken meaning
- Goal** → self-contained, semantically coherent chunks

► Chunking Strategies

Strategy	Idea	Best For
Fixed-size	Split every N tokens	Quick prototyping
With overlap	Sliding window, share boundary tokens	Avoid edge loss
Semantic	Split where meaning changes	High quality
Recursive	Try paragraph → sentence → word	Mixed content
Late chunking	Embed full doc, then chunk vectors	Long context models
LLM / Agentic	LLM decides where to split	Best quality, costly

6 Vector DB Pipeline

Docs → Chunk → Embed → Store (text + vector + metadata)

↓
Query → Embed → Semantic search → top-k chunks

- **Metadata filtering:** e.g. filter by `date>2024` or `dept=finance`

7 ANN & HNSW

- Brute force = $O(N)$ → not scalable for millions of vectors
- **ANN** (Approximate Nearest Neighbor) → fast "good enough" search
- **HNSW** = Hierarchical Navigable Small World — graph index

HNSW Intuition (multi-layer graph)

- **Top layers** → few nodes, long-range jumps
- **Bottom layers** → all nodes, local refinement

Indexing → insert each vector at random level

Retrieval → start at top → greedy walk to closest → descend → refine

Result: **$\log(N)$** search instead of N

8 Retrieval Challenges

1. **Scaling** — millions of docs

2. **Semantic gap** — query and doc use different words
3. **Retrieval precision** — top-k often noisy

9 HyDE (Hypothetical Document Embeddings)

- **Problem:** short query $\neq$ long answer document
- **Trick:** ask LLM to generate a HYPOTHETICAL answer first
- Search using THAT answer's embedding $\rightarrow$ much better match

10 Reranking

- First-stage retrieval is fast but noisy
- Add a second, slower, more accurate model to re-order top-N

<i>Bi-encoder (Retrieve)</i>	<i>Cross-encoder (Rerank)</i>
Encodes query & doc separately	Encodes (query, doc) jointly
Can pre-compute doc vectors	Must run per pair
$\swarrow$ Fast $\rightarrow$ search millions	 Slow $\rightarrow$ only top-N (~50)
Lower accuracy	Higher accuracy

11 Final RAG Pipeline

```

User Query
  ↓
Bi-encoder → retrieve top N (e.g. 50)
  ↓
Cross-encoder → rerank → top k (e.g. 5)
  ↓
LLM → grounded answer
  
```

Quick Revision Takeaway

RAG = **chunk** $\rightarrow$ **embed** $\rightarrow$ **store** $\rightarrow$ **retrieve** $\rightarrow$ **rerank** $\rightarrow$ **generate**. Bi-encoder for speed, cross-encoder for accuracy.

Topic 7 *Embeddings Deep Dive*

1 *Core Idea*

Embeddings = **numbers that capture meaning**. Similar things → close vectors. Different things → far vectors.

2 *Intuition*

- "cat" and "kitten" → vectors very close together
- "cat" and "airplane" → vectors far apart
- Computers can do **math on meaning!** 🧮

3 *Cosine Similarity*

- Measures the **ANGLE** between two vectors
- **1** → same direction (very similar)
- **0** → perpendicular (unrelated)
- **-1** → opposite direction

$$\cos(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

Tiny Worked Example

$$A = [1, 2], B = [2, 4]$$

$$A \cdot B = 1 \times 2 + 2 \times 4 = 10$$

$$||A|| = \sqrt{5}, ||B|| = \sqrt{20}$$

$$\cos = 10 / (\sqrt{5} \times \sqrt{20}) = 10/10 = \mathbf{1.0} \rightarrow \text{identical direction!}$$

4 *Word Embedding Intuition*

- Imagine a table: rows = words, columns = dimensions
- Each cell holds a learned weight
- Similar words → similar row patterns
- Famous: **king - man + woman ≈ queen** 👑



► 1. Cosine similarity

- Measures: **angle** (direction)
- Use: text/semantic search (default for NLP)
- Pitfall: ignores magnitude

► 2. Dot product

$$\mathbf{A} \cdot \mathbf{B} = \sum \mathbf{a}_i \times \mathbf{b}_i$$

- Measures: alignment + magnitude
- Use: when length matters (recommenders)
- Pitfall: longer vectors get unfair advantage

► 3. Euclidean (L_2) distance

$$\mathbf{d} = \sqrt{\sum (\mathbf{a}_i - \mathbf{b}_i)^2}$$

- Measures: straight-line distance
- Use: image features, clustering
- Pitfall: fails in very high dims (curse of dimensionality)

► 4. Manhattan (L_1) distance

$$\mathbf{d} = \sum |\mathbf{a}_i - \mathbf{b}_i|$$

- Measures: sum of axis-wise distances
- Use: sparse / high-dim data
- Pitfall: less intuitive than L_2

⚠ Normalization Note

If vectors are **L2-normalized**, cosine similarity and dot product become equivalent (up to constants). Without normalization, dot product **favors LONG vectors**.



- Vanilla BERT cross-encodes (query, doc) jointly → too slow for large search
- **SBERT = bi-encoder** → produces independent sentence embeddings
- Perfect for RAG, search, clustering, dedup

Sentence A → encoder → vector u
Sentence B → encoder → vector v
↓
Compare u and v → train via loss

- **Loss functions:**
 - Softmax (NLI: entail / contradict / neutral)
 - Triplet loss (anchor, positive, negative)
 - Contrastive loss (pull pos, push neg)
 - Multiple Negatives Ranking Loss (in-batch negatives)

7 Retrieval Evaluation Metrics

<i>Metric</i>	<i>Question</i>	<i>Use</i>
Precision@K	Of top-K, how many relevant?	Quality of top results
Recall@K	Of all relevant, how many in top-K?	Coverage
Hits@K	At least 1 relevant in top-K?	Simple binary check
MRR@K	Rank of FIRST correct answer	QA, autocomplete
MAP@K	Mean of avg precisions	Multi-relevant ranking
NDCG@K	Position-weighted relevance	Graded relevance, search

8 Fine-tuning Embeddings

- Pretrained models may fail when:
 - Domain mismatch (legal, medical, code)
 - Task mismatch (question→answer, code→doc)

9 Hard Negatives

- **Positive** → relevant doc for the query
- **Easy negative** → obviously unrelated (random doc)

- **Hard negative** → looks similar but is wrong → forces model to learn fine distinctions



Hard negatives = **the gym for rerankers**. They sharpen the decision boundary.

 Quick Revision Takeaway

Embeddings turn meaning into vectors. **Cosine for text**, normalize before dot product. **SBERT** = bi-encoder workhorse. **Hard negatives** → better rerankers.

Topic 8 Multimodal RAG

1 Core Idea

Multimodal RAG = retrieval that understands **BOTH text AND images** in the same vector space.

2 Why?

- Text-only RAG can't search images, screenshots, charts, products
- Real apps need: "find similar shoes from this photo", "show me UI bugs from this screenshot"

3 Supported Search Modes

-  →  text → text
-  →  text → image
-  →  image → image
-  →  image → text

4 CLIP (Contrastive Language-Image Pretraining)

- **Dual encoder** architecture:
 - Text encoder (Transformer)
 - Image encoder (ViT or CNN)
- Both project into the **SAME shared embedding space**
- Similar text & images sit close together

5 Training Intuition

Batch of pairs

 dog image + "a photo of a dog"

 cat image + "a photo of a cat"

 car image + "a photo of a car"

Goal:

✓ Correct pairs → **HIGH** similarity (pull together)

✗ Wrong pairs → **LOW** similarity (push apart)

→ This is **contrastive learning**

6 *Multimodal RAG Pipeline*

1. Encode all images (CLIP image encoder)
2. Encode all texts (CLIP text encoder)
3. Store all embeddings in vector DB
- ↓
4. User query (text OR image) → encode
5. Retrieve nearest items (any modality)
6. Pass to multimodal LLM → answer

7 *Practical Examples*

- "Find similar products from this image" → e-commerce
- "Show UI bugs matching this screenshot" → QA tools
- "Search slides containing this chart" → enterprise search
- "Caption this image" → accessibility

Quick Revision Takeaway

CLIP = text + image in the same vector space. Multimodal RAG = same RAG flow, but the index holds vectors from any modality.

Topic 9 *Prompt Engineering Security*

1 *Core Idea*

LLM apps add **NEW attack surfaces** (prompt injection, leaks, agent abuse) on top of normal software security.

2 *Why It Matters*

- LLMs blur the line between **code and data** → instructions can be smuggled in
- Adoption is fast 🚀 — security practices are slow 🐢
- New failure modes: prompt injection, leaks, model misuse, autonomous actions

3 *OWASP for GenAI*

- **OWASP** = Open Worldwide Application Security Project
- **GenAI Top 10** — standardized list of LLM security risks

4 *OWASP Top LLM Risks*

#	<i>Risk</i>	<i>One-line meaning</i>
1	Prompt Injection	Attacker overrides instructions via input
2	Sensitive Info Disclosure	Model leaks PII, secrets, credentials
3	Supply Chain	Tampered model / library / dataset
4	Data / Model Poisoning	Bad training data inserts hidden behavior
5	Improper Output Handling	Output executed without sanitization
6	Excessive Agency	Agent has too many permissions
7	System Prompt Leakage	Hidden instructions get exposed
8	Vector / Embedding Weakness	Poisoned docs or embedding inversion
9	Misinformation	Confident but wrong answers
10	Unbounded Consumption	Resource / cost abuse, DoS / DoW

5 *Prompt Injection*

- **Direct** → user types: "Ignore previous instructions, do X"
- **Indirect** → malicious instructions hidden in fetched webpage / doc / email
- **Unintentional** → hidden bias in inputs nudges output

Risk: model executes attacker's commands

Fix: input validation, isolate user input, hardened system prompts, output filtering

6 *Sensitive Data Disclosure*

- Risks: PII, financial, health, credentials, API keys
- **Defenses:**
 - Input sanitization & PII redaction
 - Output filtering
 - Access control on data sources
 - Adversarial PII testing

7 *Supply Chain Risk*

- Untrusted third-party libraries
- Tampered models from public hubs
- Unsafe pretrained checkpoints

8 *Data / Model Poisoning*

- Biased training datasets
- Backdoor triggers (specific phrase → bad behavior)
- Hidden malicious behavior baked in

9 *Improper Output Handling*

- Blindly executing LLM-generated SQL → SQL injection
- Rendering LLM HTML → XSS
- Running LLM-generated code → RCE
- **Fix:** treat LLM output as **UNTRUSTED**, sanitize before use

10 *Excessive Agency*

- Agent has too many tools / permissions
- **Fixes:**
 - Principle of least privilege
 - Human approval for critical actions
 - Sandboxing

11 *System Prompt Leakage*

- Attackers extract hidden system prompt
- **NEVER** store secrets / keys in prompts
- Externalize sensitive logic to backend

12 *Vector / Embedding Weakness*

- Poisoned docs added to vector DB → bad retrieval
- Embedding inversion → recover original text from vectors
- **Fix:** validate sources, secure DB, access control

13 *Misinformation*

- Confident but wrong (hallucinations)
- Critical domains → require human review + fact-checking pipelines

14 *Unbounded Consumption*

- API flooding, huge prompts, runaway loops
- **'Denial-of-Wallet'** — expensive token usage
- **Fix:** rate limits, quotas, monitoring, max-tokens caps

15 *Llama Guard*

Two-gate design (defense in depth)

Llama Guard = a safety classifier LLM

User input →  **INPUT GATE** → LLM →  **OUTPUT GATE** → User

Each gate classifies as **SAFE** or **UNSAFE** across categories:
violence, sexual, hate, self-harm, weapons, illegal, etc.

Why both? → catches malicious *inputs* AND harmful *outputs*.

Quick Revision Takeaway

Secure LLM systems need control over: **INPUTS • OUTPUTS • DATA • PERMISSIONS • USAGE**. Treat LLM output as untrusted user input.

Topic 10 *AI Agents — Foundations & Planning*

1 *Core Idea*

AI Agent = an LLM that runs in a **loop**, decides what to do next, uses tools, and pursues a goal. The LLM is the "brain".

2 *Agent vs Chatbot*

<i>Chatbot</i>	<i>Agent</i>
One reply per turn	Multi-step loop
Just talks	Acts in the world (tools)
No autonomy	Decides next step itself
No memory of state	Tracks goal & progress

3 *The Agent Loop*

Perceive → Reason → Act (tool) → Observe → Repeat

(↑ until goal is achieved or stop condition ↑)

4 *Core Characteristics*

-  LLM-driven control flow
-  Tool use (APIs, code, search)
-  Autonomy
-  Iterative loop
-  Memory (short + long term)

5 *Simple Agent Example*

User: "Book me the cheapest flight to NYC next Friday"

Agent flow:

1. Understand → "find cheapest NYC flight next Friday"
2. Action → `flight_search_api(dest='NYC', date='Fri')`
3. Observation → list of flights with prices
4. Reasoning → pick lowest price
5. Action → `book_flight(id=42)`
6. Final → "Booked flight AA123 for \$189 ✅"

6 ⚙️ Automated vs Agentic Workflow

<i>Automated Workflow</i>	<i>Agentic Workflow</i>
Deterministic, fixed path	Dynamic, adaptive path
Same input → same path	Same input → may differ
No reasoning	Reasoning + self-correction
Tools wired in advance	Tools chosen at runtime
Developer controls path	Model controls path
Predictable, cheap	Flexible, costlier

7 🪜 Complexity Ladder

Single LLM call → LLM + RAG → Workflow (fixed) → Agent (dynamic)

👉 Golden Rule

Start with the **SIMPLEST** solution that works. Add complexity **only when it measurably helps**.

8 ⚠️ Tradeoffs

- More autonomy → more latency, cost, error surface
- Workflows → predictable tasks
- Agents → open-ended tasks with unknown # of steps

- Always add: **guardrails, sandboxing, human checkpoints, stop conditions**

9 *How to Build an Agent (6 steps)*

1. Define **real scenarios** (concrete user tasks)
2. Write **SOP** — how a smart human would do it
3. Build **MVP** (single prompt → simple loop)
4. Connect APIs / data / orchestration
5. Test manually + automatically
6. Deploy + iterate from real usage

10 *Why Agents Fail*

Most agents fail because **reasoning is weak**.

👉 Prove a **SINGLE prompt** can reason well **BEFORE** building multi-step orchestration.

11 *SOP Test*

"If I gave this task to a smart intern, what steps would they follow?"

If you can't answer this clearly → the task is too broad to automate.

12 *Cheat Sheet*

Type	Reasoning	Tools	Best For
Chatbot	Minimal	None	Q&A, conversation
LLM + RAG	Light	Retrieval	Knowledge lookup
Workflow	Fixed	Wired	Predictable pipelines
Agent	Dynamic	Chosen	Open-ended goals

Quick Revision Takeaway

Agents = **LLM in a loop with tools**. Use only when workflows can't handle the variability. Prove reasoning first, orchestrate second.

1 Top Definitions

- **AI Engineering** → build apps using pretrained foundation models
- **Self-attention** → each token attends to others via Q,K,V
- **Causal mask** → blocks future tokens (decoder-only)
- **Decoding** → how to pick next token from probabilities
- **RAG** → retrieve docs + augment prompt + generate
- **Embedding** → vector that represents meaning
- **CLIP** → text + image in same vector space
- **Prompt injection** → attacker overrides instructions via input
- **Agent** → LLM in a loop using tools to reach a goal
- **LLM-as-Judge** → use a strong LLM to grade outputs

2 Key Formulas

$$\text{Attention}(Q,K,V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

$$\text{softmax}(\text{logits} / T)$$

low T → sharp, high T → flat

$$\cos(A,B) = (A \cdot B) / (||A|| \cdot ||B||)$$

$$\text{Euclidean: } d = \sqrt{\sum (a_i - b_i)^2}$$

3 High-Yield Comparisons

<i>Concept</i>	<i>Option A</i>	<i>Option B</i>
ML vs AI Eng	Train models	Use pretrained models
Enc-Dec vs Decoder	Seq2seq, BERT/T5	GPT, modern LLMs
Bi vs Cross encoder	Fast retrieve	Slow rerank
BLEU vs ROUGE	Precision	Recall
CoT vs ReAct	Reason only	Reason + tools
Workflow vs Agent	Fixed path	Dynamic path
Prompt vs RAG vs FT	Cheap, limited	Updateable / Permanent

4 *Critical Flows*

LLM → tokenize → embed → +pos → N×blocks → softmax → token

RAG → query → embed → retrieve top-N → rerank → top-k → LLM

ReAct → Thought → Action → Observation → ... → Answer

Eval → dataset → system → outputs → metrics → iterate

Agent → Perceive → Reason → Act → Observe → loop

5 *Most Important Pitfalls*

- Greedy decoding → repetitive, dull
- Chunks too big → noisy retrieval; too small → broken meaning
- Dot product without normalization → favors long vectors
- Vanilla BERT cross-encode → too slow for large search → use SBERT
- LLM output is **UNTRUSTED** → never execute blindly
- System prompts are **NOT secret** → never store keys there
- Agents fail when reasoning is weak — fix the prompt first
- More autonomy = more cost + more failure modes
- BLEU/ROUGE miss meaning → use BERTScore / LLM-as-judge
- Hallucination = confident + wrong → ground with RAG

6 *One-Line Memory Hooks*

- **T1** — AI Eng = orchestrate, don't train.
- **T2** — Decoder-only = next-token + causal mask.
- **T3** — Temperature = creativity dial.
- **T4** — Eval = offline + online, both required.
- **T5** — Simple → CoT → ReAct → RAG.
- **T6** — RAG = chunk-embed-retrieve-rerank-generate.
- **T7** — Cosine for text, normalize for dot product.
- **T8** — CLIP = one space, two modalities.
- **T9** — Treat LLM output like untrusted user input.
- **T10** — Agent = LLM + loop + tools + goal.

7 *Final Exam Mantra*

- ♦ Start simple. Measure everything. Ground with RAG. Sanitize all I/O.
- ♦ Pick the smallest tool that works. Use agents only when needed.
- ♦ **Quality of reasoning > complexity of orchestration.**